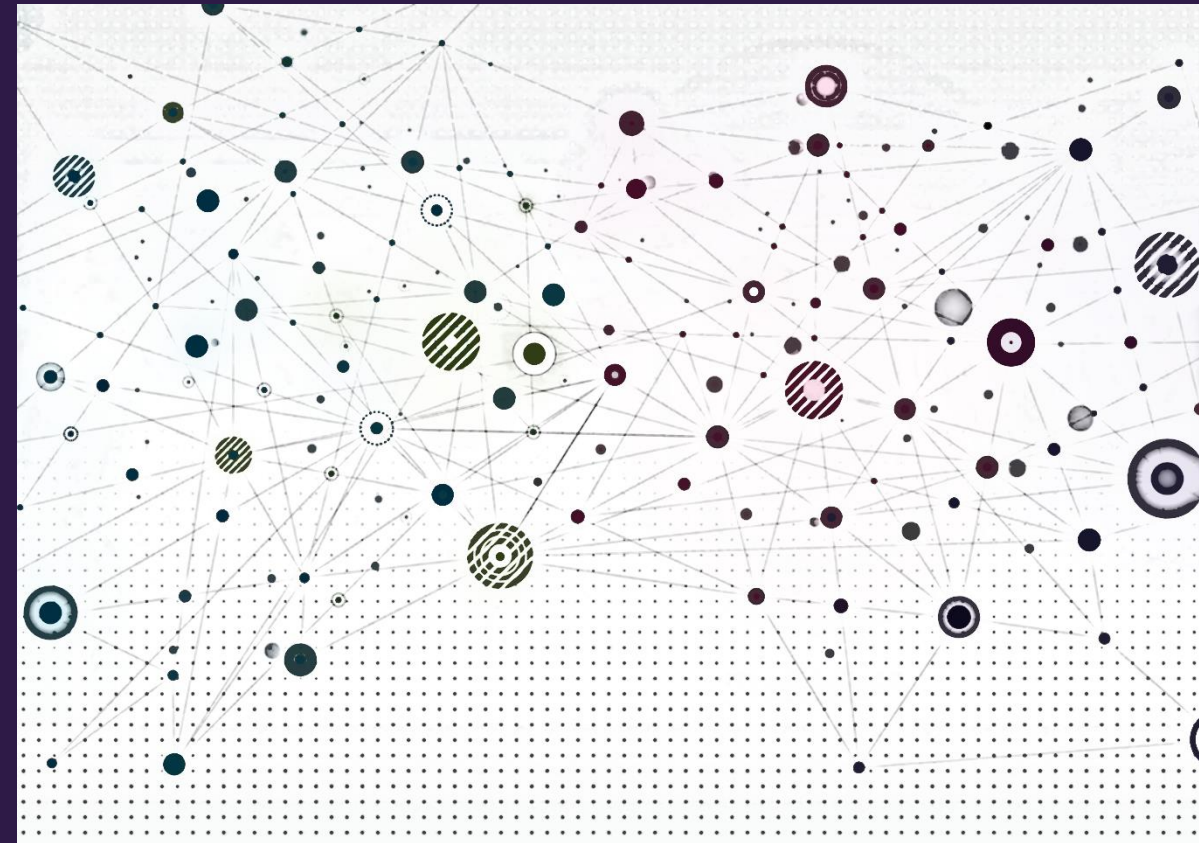


Classical Graph Representations and Properties





Overview

- **Attendance**
- Adjacency matrix
- Node degree and network density (sparsity)
- Graph Laplacian
- Walks and paths
- Centrality measures
- Node groups
- Node similarity metrics



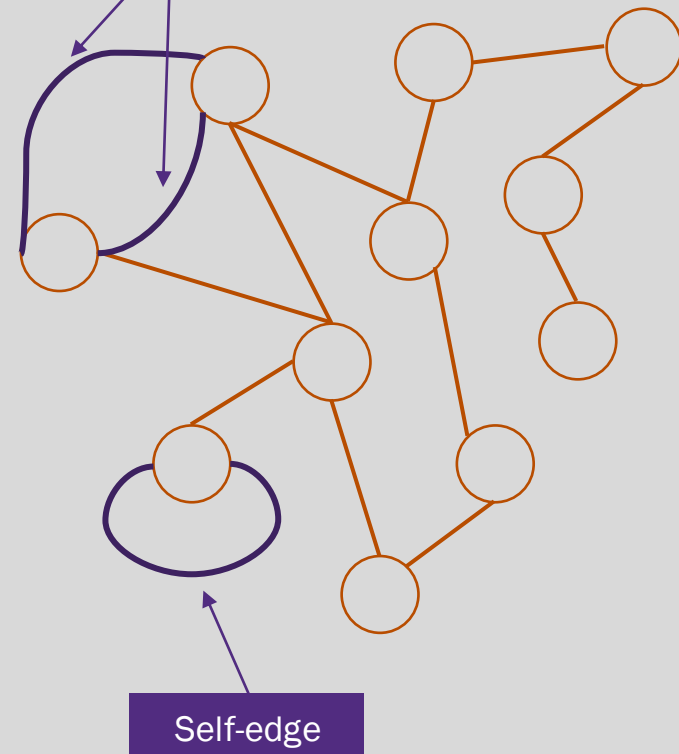
Adjacency matrix





What is a graph?

- A graph, $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, is defined by a set of nodes, $\mathcal{V}$, and a set of edges, $\mathcal{E}$. An edge from node $u \in \mathcal{V}$ to node $v \in \mathcal{V}$ is denoted as $(u, v) \in \mathcal{E}$.
- n denotes the number of nodes
- m denotes the number of edges
- *multiedge* occurs when there are multiple edges between the same node
- *Self-edge* (or self-loop) occurs when a node is connected to itself



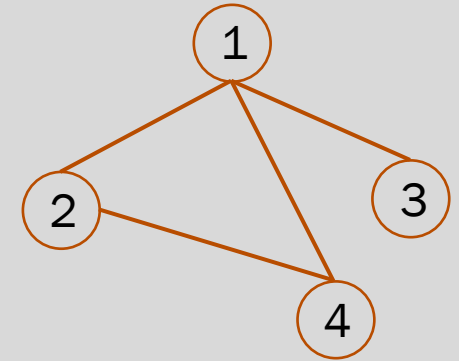


Adjacency matrix for a simple undirected network

- Consider a “simple” undirected network with unweighted edges and no self-loops, and no multi-edges
- Label the nodes with integers $1 \cdots n$
- The **adjacency matrix**, $\mathbf{A}$, is the $n \times n$ matrix with elements a_{ij}

$$a_{ij} = \begin{cases} 1 & \text{if there is an edge between nodes } i \text{ and } j \\ 0 & \text{otherwise} \end{cases}$$

- Diagonal elements are all zero (no self-loops)
- Matrix is symmetric, $a_{ij} = a_{ji}$

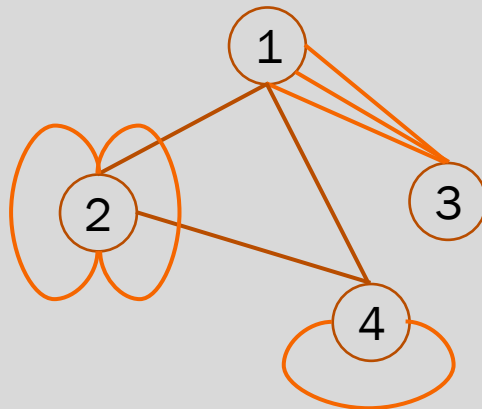


	1	2	3	4
1	0	1	1	1
2	1	0	0	1
3	1	0	0	0
4	1	1	0	0



Adjacency matrix with self-loops and multiedges

- Multiedge is represented by setting a_{ij} and a_{ji} equal to the multiplicity of the edge
 - Example: a triple edge between nodes i and j is represented by $a_{ij} = a_{ji} = 3$
- Self-edge (node connected to itself)
 - *Single self-edge is represented by setting $a_{ii} = 2$. Why not 1?
 - Multiple self-edges are represented by setting $a_{ii} = 2 \times$ multiplicity (i.e., twice the number of self-edges on node i)



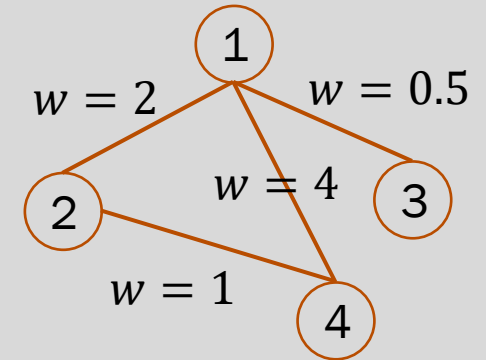
	1	2	3	4
1	0	1	3	1
2	1	4	0	1
3	3	0	0	0
4	1	1	0	2

*In directed networks, self-edges are indicated by $a_{ii} = 1$ (see slide 9)



Adjacency matrix on weighted networks

- Weights may be assigned to edges to indicate the “strength” of the connection
 - Data flow capacity between networked devices
 - Energy flow in a physical system
- Weights are typically positive but not required (e.g., social networks with adversarial relationships)
- Weights are represented in the adjacency matrix by setting $a_{ij} = a_{ji} = w$ where w is the weight of the edge between i and j
- Adjacency matrix is still symmetric for an undirected graph



	1	2	3	4
1	0	2	0.5	4
2	2	0	0	1
3	0.5	0	0	0
4	4	1	0	0

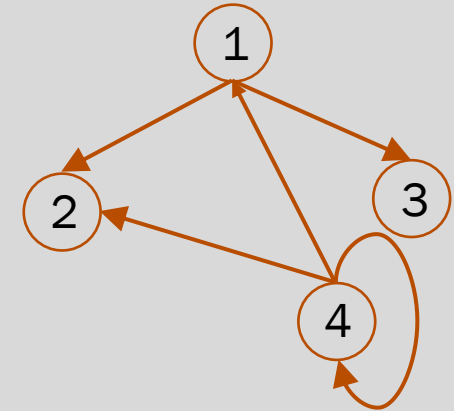


Adjacency matrix on directed networks

- Edges in a directed network are one-way connections
 - A directed edge from node i to j does not imply an edge from j to i
- The **adjacency matrix**, $\mathbf{A}$, of a directed network is the $n \times n$ matrix with elements a_{ij}

$$a_{ij} = \begin{cases} 1 & \text{if there is an edge from } j \text{ to } i \\ 0 & \text{otherwise} \end{cases}$$

- Adjacency matrix of a directed network is generally asymmetric, $a_{ij} \neq a_{ji}$
- Self-edges (unweighted) are represented by a 1 (not 2 as in undirected network)
- Rules for multi-edges and weighted edges are the same as undirected networks noting that $a_{ij} > 0$ only for the directed edges from j to i



	1	2	3	4
1	0	0	0	1
2	1	0	0	1
3	1	0	0	0
4	0	0	0	1



Node degree & network density (sparsity)





Node degree for undirected networks

- The degree, k_i , of node i in an undirected network is the number of edges incident to i
 - NOT the number of nodes connected to it
 - A multiedge adds its multiplicity to the node degree

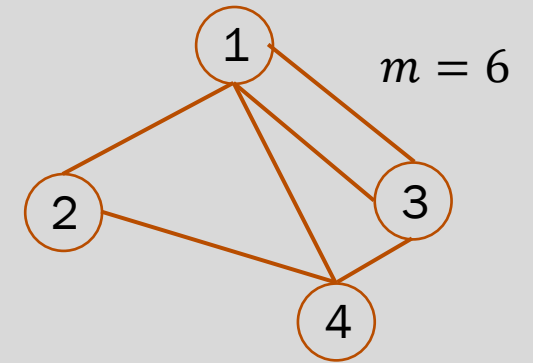
$$k_i = \sum_{j=1}^n a_{ij}$$

- An undirected network with m edges has $2m$ edge endpoints (i.e., points incident on a node) and hence

$$2m = \sum_{j=1}^n k_i = \sum_{ij} a_{ij}$$

- The mean degree over all nodes in the network is

$$c = \frac{2m}{n}$$



Node	Degree
1	4
2	2
3	3
4	3

$$\sum_{j=1}^4 k_i = 12 = 2m$$

$$c = \frac{12}{4} = 3$$



Node degree for directed networks

- Two degree types for each node in a directed network
 - in-degree* is the number of edges entering the node
 - out-degree* is the number of edges leaving the node
- Recalling that $a_{ij} = 1$ in the adjacency matrix of a directed network if there is an edge from j to i , then

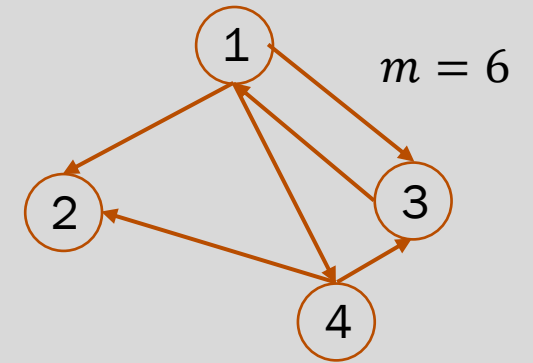
$$k_i^{in} = \sum_{j=1}^n a_{ij} \text{ and } k_i^{out} = \sum_{j=1}^n a_{ji}$$

- The total number of edges, m , is equivalently given by any of

$$m = \sum_{i=1}^n k_i^{in} = \sum_{j=1}^n k_j^{out} = \frac{1}{2} \sum_{ij} a_{ij}$$

- The mean *in-degree* is always equal to the mean *out-degree*

$$c_{in} = c_{out} = c = \frac{m}{n}$$



Node	<i>In-degree</i>	<i>out-degree</i>
1	1	3
2	2	0
3	2	1
4	1	2

$$\sum_{i=1}^4 k_i^{in} = \sum_{j=1}^4 k_j^{out} = 6$$

$$c = \frac{6}{4} = 1.5$$



Network density

- The *maximum* number of edges in a simple, undirected network with n nodes is $m_{max} = \binom{n}{2} = \frac{n(n-1)}{2}$ (Why?)
- The *density*, ρ , of the network is the fraction of m_{max} edges that are present. That is for a network with m edges:

$$\rho = \frac{m}{m_{max}} = \frac{2m}{n(n-1)} = \frac{c}{n-1} \approx \frac{c}{n}$$

- Probability that a pair of randomly selected nodes is connected
- *Dense network* – if the network density, ρ , remains non-zero as the number of nodes, n increases, it is considered *dense*
- *Sparse network* – if the network density, ρ , approaches zero as the number of nodes, n increases, it is considered *sparse*



Network density and growth

- In a dense network where ρ is constant as $n \rightarrow \infty$, the average degree, c , grows linearly with n
 - Recall for large n , that $\rho = \frac{c}{n}$
 - To remain constant, we require $c = \rho n \rightarrow m = \frac{\rho n^2}{2}$ (number of edges grows quadratically with n)
- In a sparse network, the average degree, c , can grow
 - sublinearly (sparse) OR
 - remains constant in which case $\rho \rightarrow \frac{1}{n}$ (extremely sparse)

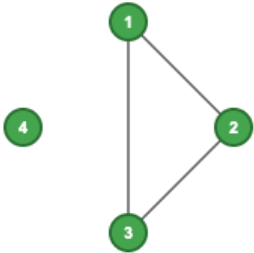
Edge growth in a dense network $\rho = 1/2$

Graph with 4 nodes

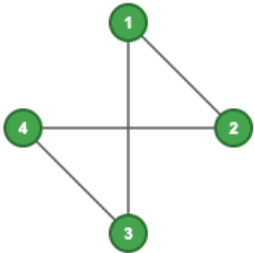
Exact Formula
Edges: 3
Density: 0.500

Asymptotic Formula
Edges: 4
Density: 0.667
Error: 33.3%

Exact



Asymptotic

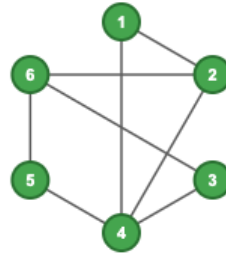


Graph with 6 nodes

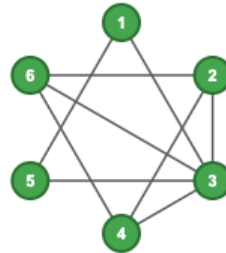
Exact Formula
Edges: 8
Density: 0.533

Asymptotic Formula
Edges: 9
Density: 0.600
Error: 12.5%

Exact



Asymptotic

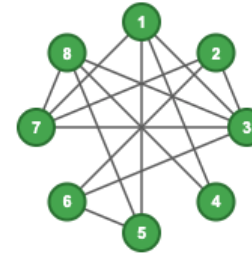


Graph with 8 nodes

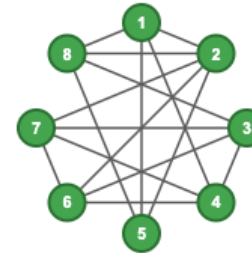
Exact Formula
Edges: 14
Density: 0.500

Asymptotic Formula
Edges: 16
Density: 0.571
Error: 14.3%

Exact



Asymptotic



Graph with 10 nodes

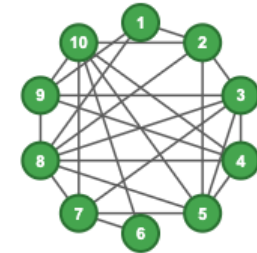
Exact Formula
Edges: 23
Density: 0.511

Asymptotic Formula
Edges: 25
Density: 0.556
Error: 8.7%

Exact



Asymptotic

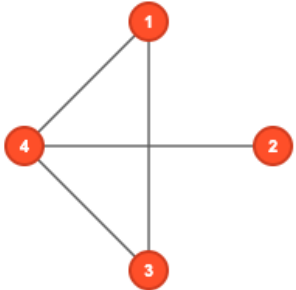


Edge growth in a sparse network $c = 2$

Sparse Graph: 4 nodes

Network Properties:

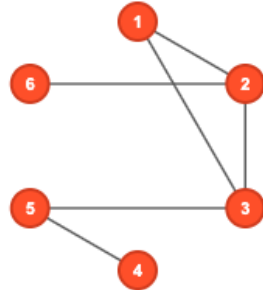
Nodes (n): 4
 Edges (m): 4
 Max possible edges: 6
 Edge density (p): 0.500
 Average degree: 2.0
 $p = c/n = 2/4 = 0.500$



Sparse Graph: 6 nodes

Network Properties:

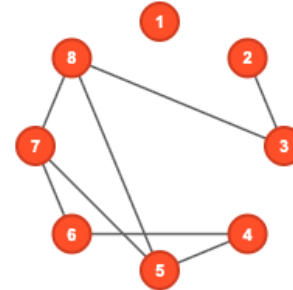
Nodes (n): 6
 Edges (m): 6
 Max possible edges: 15
 Edge density (p): 0.333
 Average degree: 2.0
 $p = c/n = 2/6 = 0.333$



Sparse Graph: 8 nodes

Network Properties:

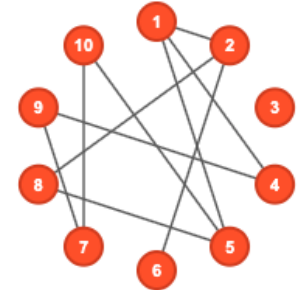
Nodes (n): 8
 Edges (m): 8
 Max possible edges: 28
 Edge density (p): 0.250
 Average degree: 2.0
 $p = c/n = 2/8 = 0.250$



Sparse Graph: 10 nodes

Network Properties:

Nodes (n): 10
 Edges (m): 10
 Max possible edges: 45
 Edge density (p): 0.200
 Average degree: 2.0
 $p = c/n = 2/10 = 0.200$



https://ml4g.masinolab.com/topics/network-metrics/sparse_network_constant_degree.html



Graph Laplacian





Graph Laplacian

- The graph Laplacian, $\mathbf{L}$, of a **simple undirected network** is an $n \times n$ matrix with elements

$$l_{ij} = k_i \delta_{ij} - a_{ij} = \begin{cases} k_i & \text{if } i = j \\ -1 & \text{if } i \neq j \text{ and there is an edge from } i \text{ to } j \\ 0 & \text{otherwise} \end{cases}$$

where k_i is the degree of node i and δ_{ij} is the Kronecker delta (1 if $i = j$ and 0 otherwise)

- Matrix form $\mathbf{L} = \mathbf{D} - \mathbf{A}$ where

$$\mathbf{D} = \begin{pmatrix} k_1 & 0 & \cdots \\ 0 & k_2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}$$

- For **weighted networks**, substitute the weighted adjacency matrix and $k_i = \sum_j a_{ij}$
- No extensions for networks with self-edges or directed networks



Graph Laplacian properties

- Every row (and column) of the graph Laplacian, $\mathbf{L}$, sums to zero

$$\sum_j l_{ij} = \sum_j (k_i \delta_{ij} - a_{ij}) = k_i - k_i = 0$$

- All eigenvalues are ≥ 0
- There is always at least one zero eigenvalue - for a connected graph there is exactly one, otherwise there is one zero eigenvalue for each component
- The *algebraic connectivity* or *spectral gap* is the value of the smallest non-zero eigenvalue



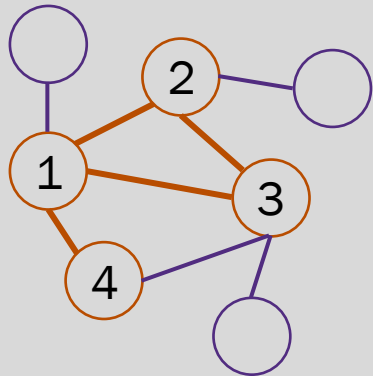
Walks and paths



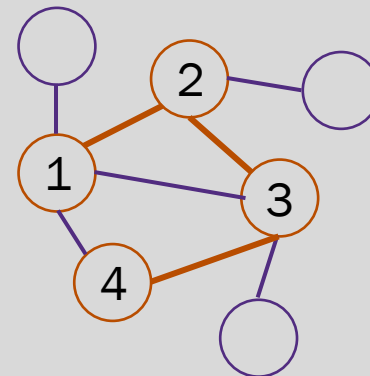
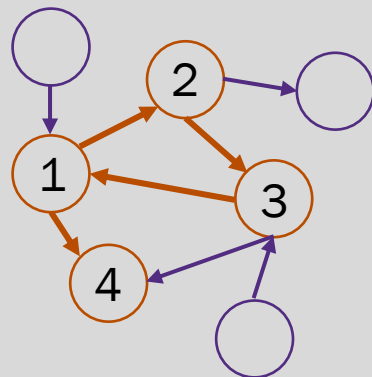


Walks and paths

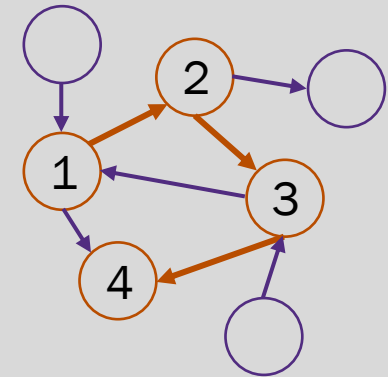
- A *walk* is any sequence of nodes such that every consecutive pair is connected by an edge
 - In a directed network, each edge in a walk must follow the edge direction
- A *path* is a *walk* that does not intersect itself (i.e., all nodes in a path are distinct)



The walk $\{(1,2), (2,3), (3,1), (1,4)\}$ traverses the network from node 1 to 4. It intersects itself at node 1 and 4, is, therefore, **not a path**.



The walk $\{(1,2), (2,3), (3,4)\}$ traverses the network from node 1 to 4. It does **not** intersect itself. **This walk is also a path.**



Network walks & paths

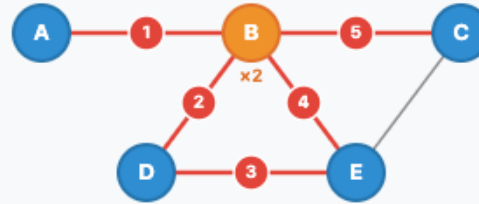
- Walk – sequence of nodes where consecutive pairs are connected by edges
- Path – a walk where all nodes are distinct

Legend

- Regular node
- Repeated node
- Highlighted sequence

Undirected Network - Walk

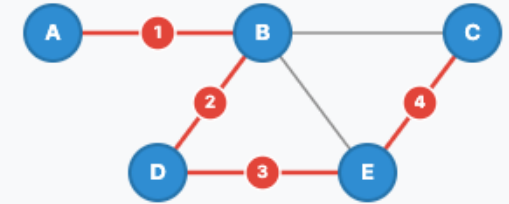
A→B→D→E→B→C



WALK: Node B visited twice, forming a loop B→D→E→B

Undirected Network - Path

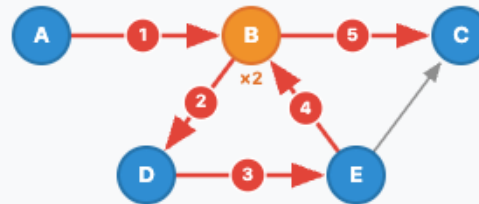
A→B→D→E→C



PATH: All nodes distinct, no repetition

Directed Network - Walk

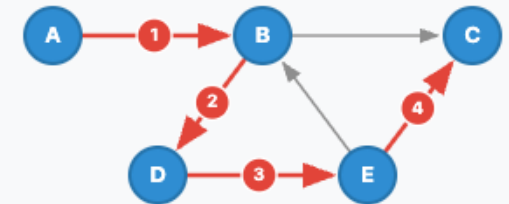
A→B→D→E→B→C



WALK: Node B visited twice, edges follow direction

Directed Network - Path

A→B→D→E→C



PATH: All nodes distinct, edges follow direction



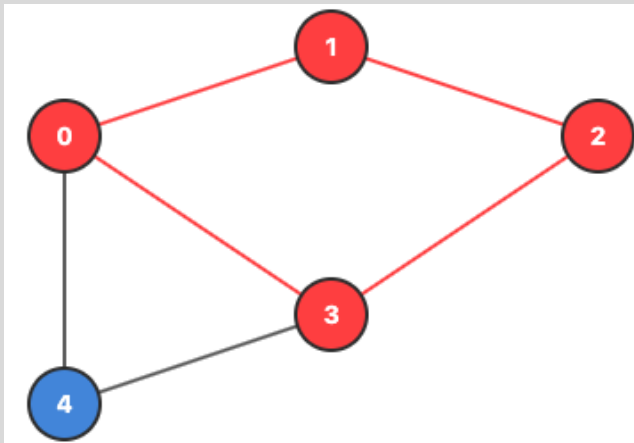
Walk length and walk counts

- The walk length is the number of edges (not nodes) in the walk
- The total number of walks of length r between nodes i and j is

$$w_{ij}^{(r)} = [A^r]_{ij}$$

where $[A^r]_{ij}$ is the element a_{ij} in the adjacency matrix to the r^{th} power

- Note, walks have direction even in undirected networks. Walk $1 \rightarrow 2$ is distinct from $2 \rightarrow 1$



3	0	2	1	1
0	2	0	2	1
2	0	2	0	1
1	2	0	3	1
1	1	1	1	2

Walks of length two from node 0 to 2 are highlighted in red. The corresponding value of $[A^2]_{0,2}$ is equal to 2 indicating these are the only two walks (they happen to be paths).



Shortest paths

- A *shortest path* is the shortest walk (i.e., traverses the fewest edges) between a pair of nodes
 - Such a walk is necessarily a path (Why?)
- The *network diameter* is the length of the longest shortest path (i.e., among all shortest paths between every pair of nodes, the diameter is the longest of these)
- Knowledge of shortest path distributions can inform graph neural network design (e.g., how many layers) (future lecture)

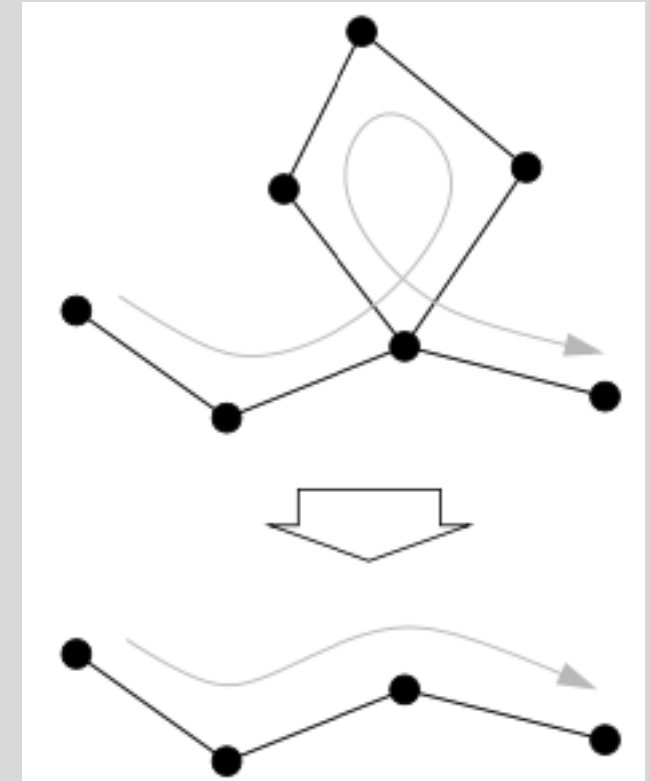


Image credit: Newman 2018



Independent paths and connectivity

- *Edge independent paths* – two paths connecting nodes i and j are **edge** independent if they share no edges
- *Node independent paths* - two paths connecting nodes i and j are **node** independent if they share no nodes
- *Edge connectivity* is the number of edge independent paths between a pair of nodes.
- *Node connectivity* is the number of node independent paths between a pair of nodes.

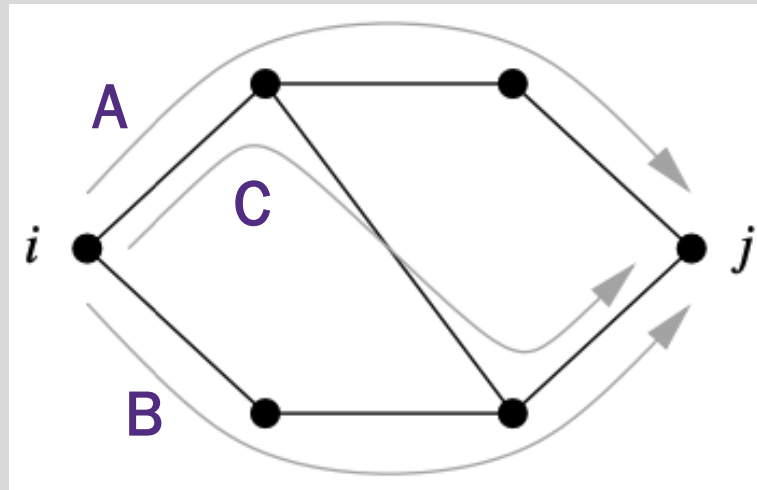


Image credit: Newman 2018

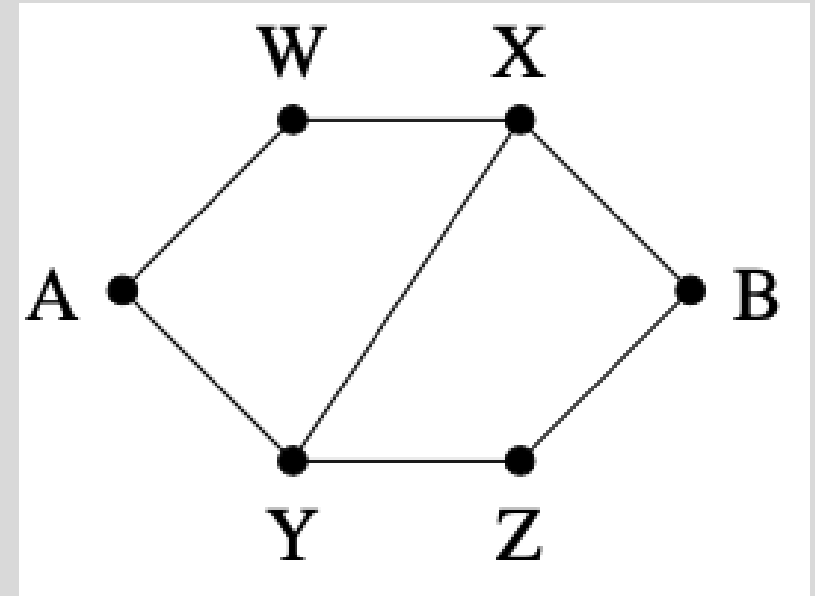
- Paths A and B are node and edge independent from each other
- Path C is NOT node or edge independent from paths A or B
- Node and edge connectivity between i and j is 2



Bottlenecks and cut sets

- A *node cut set* is a set of nodes whose removal will disconnect a specified pair of nodes
 - A cut set with a single node is a bottleneck
 - We'll see later that bottlenecks can be a problem in GNNs
- An *edge cut set* is a set of edges whose removal will disconnect a specified pair of nodes
- *Minimum cut set* is the smallest cut set that will disconnect a pair of nodes
 - *Menger's theorem*: the minimum cut set is equal to the number of independent paths between the nodes

Image credit: Newman 2018



- Two independent paths from A to B: {A, W, X, B}, and {A, Y, Z, B}
- Minimum cut sets are {W, Y}, {X, Z}, {X, Y}
- Why isn't {W, Z}?



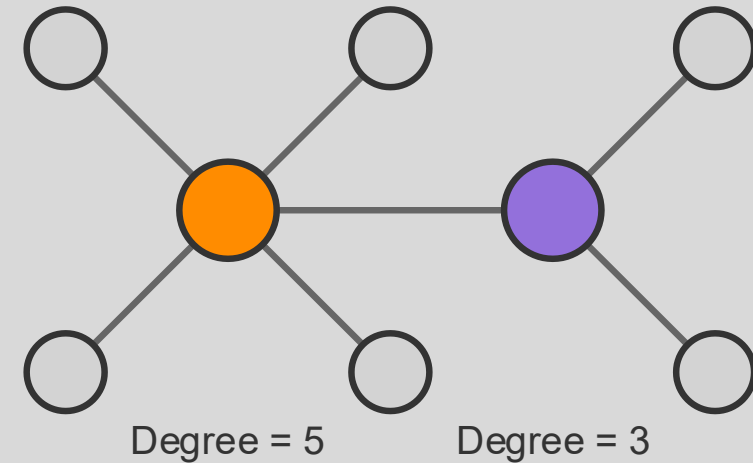
Centrality measures





Centrality concept

- Centrality metrics are intended to measure the importance of a node in a network
- Node degree can be considered a centrality measure where nodes with higher degree are considered more important
 - Doesn't consider importance of neighbors
 - Doesn't consider role in network structure
- There are many others to discuss





Eigenvector centrality for undirected networks

- In many networks, a node is important if it is connected to other important nodes
- Consider an undirected network. Let's define the centrality, x_i , of node i to be *proportional* to the centralities of its neighbors

$$x_i = \kappa^{-1} \sum_{j=1}^n a_{ij} x_j$$

where κ^{-1} is the inverse proportionality and a_{ij} is the i, j element of the adjacency matrix

- So we need the centrality measures to compute the centrality measures? How do we proceed?



Eigenvector centrality for undirected networks

- We can rewrite $x_i = \kappa^{-1} \sum_{j=1}^n a_{ij} x_j$ in matrix form $\mathbf{x} = \kappa^{-1} \mathbf{A} \mathbf{x}$ which is equivalent to **eigenvalue problem**:

$$\mathbf{A} \mathbf{x} = \kappa \mathbf{x}$$

- Select the leading eigenvector of $\mathbf{A}$ (the one corresponding to the largest positive eigenvalue)
- Set κ to the value value of the leading eigenvalue
- Centrality x_i of node i is the i^{th} element of the leading eigenvector

Linear Algebra for the win!!

Linear algebra is fundamental to machine learning. Graph learning is no exception. If you want a linear algebra refresher, I suggest Dr. [Gilbert Strang's MIT's open course](#).

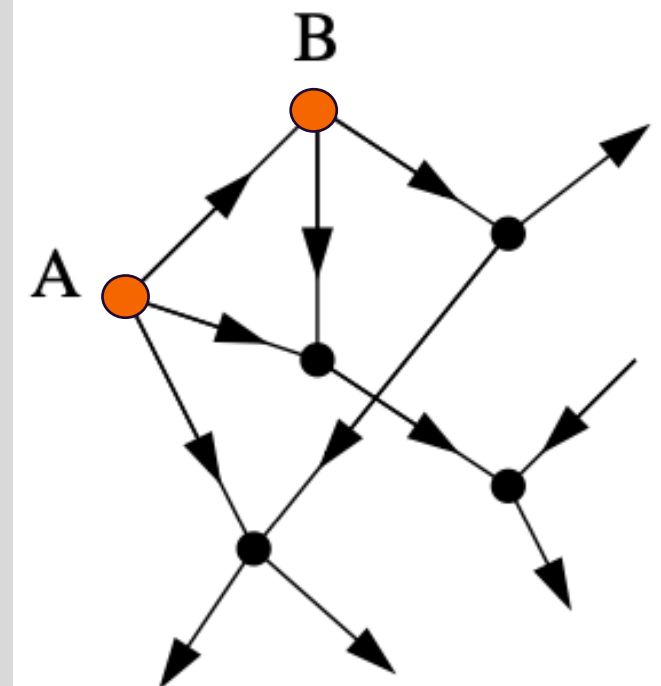




Eigenvector centrality on directed networks

- Adjacency matrix of a directed network generally has *left* and *right* eigenvectors
- The *right* eigenvector indicates the importance of node i based on the importance of nodes that point to it.
- Limitations
 - Only nodes in a strongly connected component of two or more nodes (or its out component) will have non-zero eigenvector centrality
 - A node could have a high in-degree but zero eigenvector centrality

Image credit: Newman 2018



- Node A has zero eigenvector centrality because it has no nodes pointing to it
- Node B is pointed to by node A only, so it also has zero eigenvector centrality



PageRank

- Addresses limitations of eigenvector centrality on directed networks by
 - Adding a small constant importance, β , to **every** node
 - Scaling the contribution of neighbor nodes importance by the inverse of their out-degree

$$x_i = \alpha \sum_j a_{ij} \frac{x_j}{k_j^{out}} + \beta$$

- In matrix form (setting $\beta = \mathbf{1}$)

$$\mathbf{x} = \alpha \mathbf{A} \mathbf{D}^{-1} \mathbf{x} + \mathbf{1} = (\mathbf{I} - \alpha \mathbf{A} \mathbf{D}^{-1})^{-1} \mathbf{1}$$

- The scaling term α must be chosen carefully.
 - If it's too small, every node's PageRank approaches β
 - If it's too large, $\mathbf{I} - \alpha \mathbf{A} \mathbf{D}^{-1}$ becomes singular
 - Undirected networks choose $\alpha < 1$, directed network requires experimentation



- PageRank was developed by Google for web page ranking
- Still plays a central role in Google search, though other factors are used
- $\alpha=0.85$ – no one knows how this was selected!

Betweenness Centrality

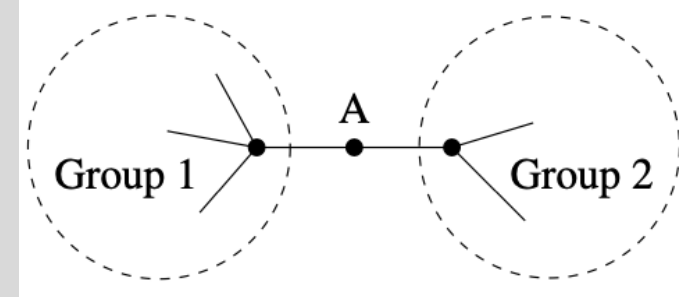
- Measures the extent to which a node is on shortest paths between other nodes
- The betweenness centrality, x_i , of node i is

$$x_i = \sum_{st} \frac{n_{st}^i}{g_{st}}$$

where the sum is over all shortest paths for all node pairs $\{s, t\}$, n_{st}^i is the number of shortest paths from s to t that contain i , and g_{st} is the number of shortest paths from s to t

- Removal of nodes with high betweenness centrality will most disrupt "flow" (of information, water, energy, etc.)
- May be convenient to normalize by $1/n^2$
- Can be applied to directed and undirected networks

Image credit: Newman 2018



- Node A has low degree (only 2 neighbors)
- All paths between nodes in Group 1 and those in Group 2 must pass through A
- A will have high betweenness centrality



Node groups

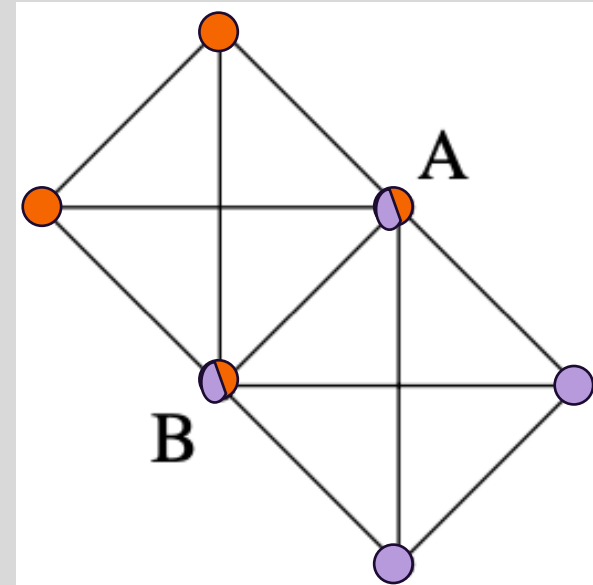
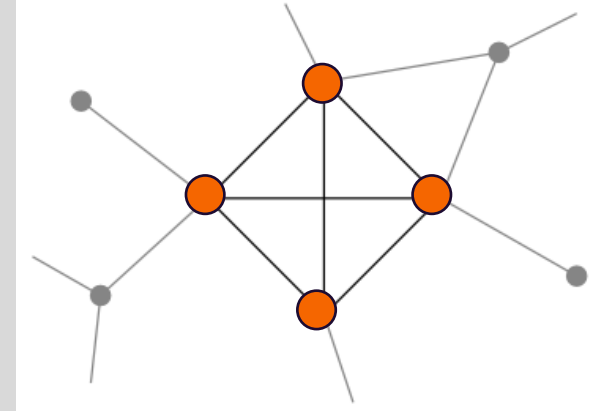




Cliques

- A *clique* is a set of nodes in which every node is connected to every other node within an undirected network
- Cliques may overlap, i.e., some nodes in one clique may belong to another clique
- A clique found in a sparse network may indicate that the clique nodes are more similar than randomly selected nodes
 - Family members in a social group
 - Groups of interacting proteins in cellular networks
 - Financial entities that interact through transactions

Image credit: Newman 2018



- A clique of 4 nodes (top)
- Two cliques with overlapping nodes (bottom)



***k*-cores**

- A *k*-core is a set of connected nodes (every node is reachable from every other node) where each node is linked to at least *k* of the other nodes
- Less stringent grouping than a clique
- *k*-cores are nested, i.e., the nodes in the *k*-core are in the nodes in the (*k*-1)-core

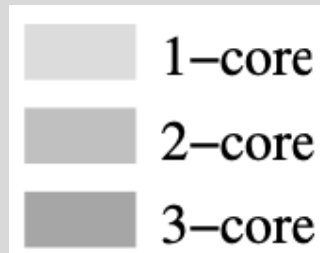
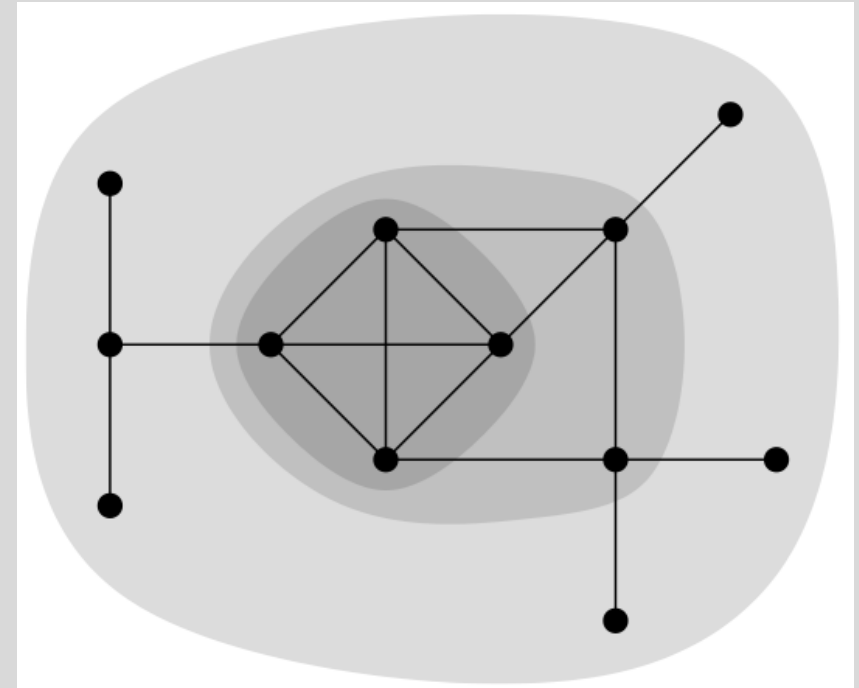


Image credit: Newman 2018



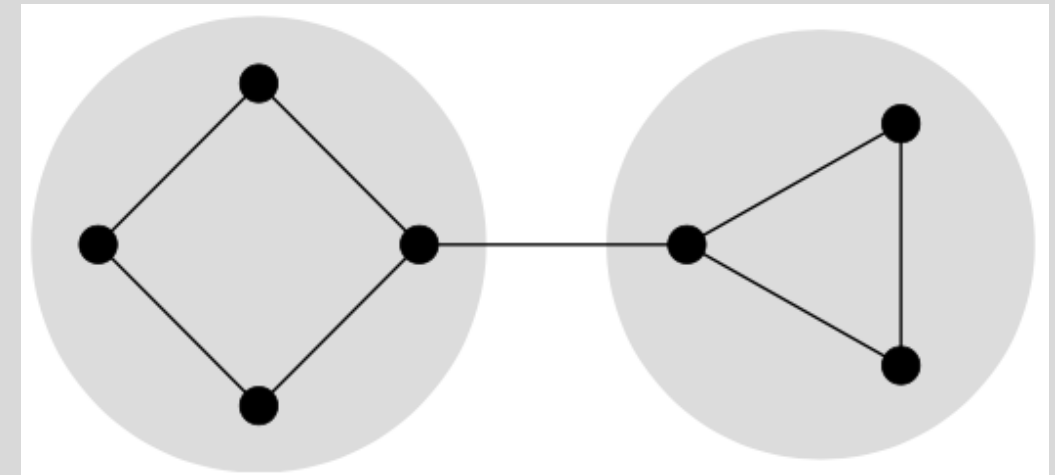
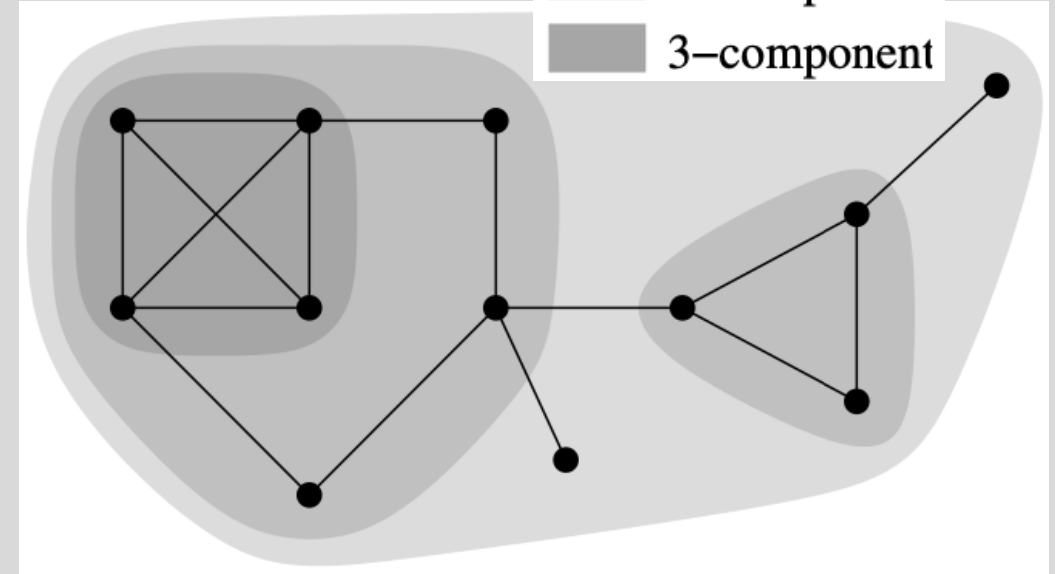
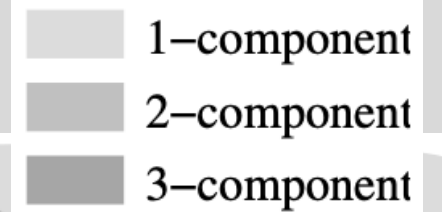
- All nodes in the network together form the 1-core
- The 2-core nodes are a subset of the 1-core nodes
- The 3-core nodes are a subset of the 2-core nodes
- No cores with $k > 3$



k -components

- A k -component is a set of nodes where every node is reachable from every other node by at least k **node-independent** paths
- No pair of nodes can be disconnected without removing less than k other nodes
- k -components are nested like k -cores
- Distinct from k -cores

Image credit: Newman 2018



- K -components are different than k -cores
- (Bottom) Two 2-components (shaded) and one 2-core (entire network)



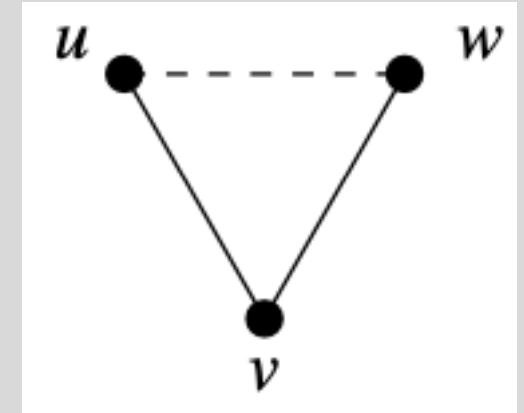
Clustering coefficient

- Consider an undirected network in which node u is connected to node v and node v is connected to node w
 - Yields the 2-edge path $u \rightarrow v \rightarrow w$
 - If u is also connected to w we say the path is closed
- The *clustering coefficient*, C , is the fraction of all such paths of length two that are closed

$$C = \frac{\text{Number of closed paths of length two}}{\text{number of paths of length two}}$$

$$C = \frac{(\text{number of triangles}) \times 6}{\text{number of paths of length two}}$$

Image credit: Newman 2018



- Social network studies have found clustering coefficients around 0.1
- Random connections in the studied networks would yield clustering coefficients < 0.001
- Suggests two people will be connected if they have a shared connection

<https://ml4g.masinolab.com/topics/network-metrics/clustering-coefficient-demo.html>



Homophily

- *Homophily* in networks is the tendency of nodes to be connected to similar nodes
- Also called assortative mixing
- Often based on node features, e.g., class membership
- *Degree homophily* (assortative mixing by degree, i.e., nodes are connected to nodes with similar degree) is derived from network structure and is defined by

$$\frac{1}{2m} \sum_{ij} \left(a_{ij} - \frac{k_i k_j}{2m} \right) k_i k_j$$

which is the covariance of node degree across all node pairs in the network



Node similarity metrics





Node similarity

- Here, we focus on similarity measures that only consider network structure.
- Later, we'll see methods that incorporate node features



Cosine similarity

- Consider two vectors $\mathbf{x}$ and $\mathbf{y}$. The cosine of the angle, θ , between them is

$$\cos\theta = \frac{\mathbf{x} \cdot \mathbf{y}}{|\mathbf{x}||\mathbf{y}|}$$

- Consider an undirected, unweighted network. Let the i^{th} and j^{th} rows, $\mathbf{a}_i$ and $\mathbf{a}_j$, of the adjacency matrix be vectors, their cosine similarity, σ_{ij} , is

$$\sigma_{ij} = \frac{\mathbf{a}_i \cdot \mathbf{a}_j^T}{|\mathbf{a}_i||\mathbf{a}_j|} = \frac{n_{ij}}{\sqrt{k_i k_j}}$$

where n_{ij} is the number of shared nodes between nodes i and j

- Structural equivalence metric
- Value lies in $[0,1]$ (assuming non-negative vector values)
- Widely used in machine learning to compare embedding vector similarity



Other structural equivalence similarity metrics

- Jaccard coefficient

$$J_{ij} = \frac{n_{ij}}{k_i + k_j - n_{ij}}$$

- Pearson correlation coefficient

$$r_{ij} = \frac{(\mathbf{a}_i - \bar{\mathbf{a}}_i) \cdot (\mathbf{a}_j - \bar{\mathbf{a}}_j)}{\sqrt{(\mathbf{a}_i - \bar{\mathbf{a}}_i)^2} \sqrt{(\mathbf{a}_j - \bar{\mathbf{a}}_j)^2}}$$

Where $\bar{\mathbf{a}}_j$ is the mean of row j

